Kasi Yedumati · Full Stack Training

Lesson 7 — Project Structure

Course: FastAPI for Full Stack · **Session 7 of 10** · Learning path · **Previous:** Lesson 6 · **Next:** Lesson 8

> **Module Outcome:** You can split the employee API into files, hand each route a database session with `Depends`, read the database URL from the environment, and run one automated test.

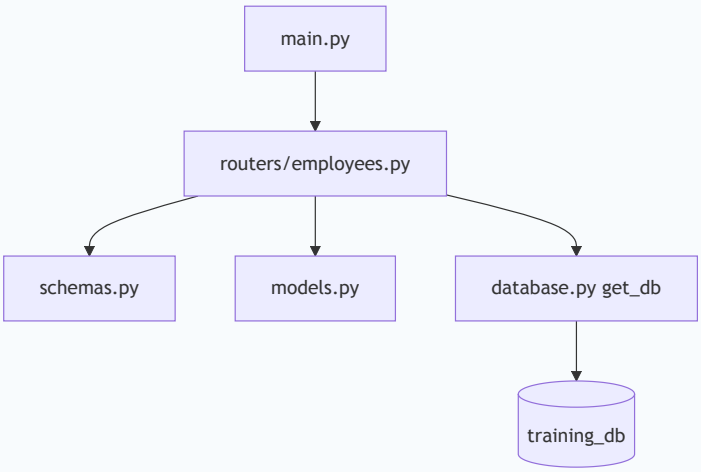
You will build	The employee API as a small project
New words	Router, dependency, environment variable
You need	The employee CRUD API running against `training_db`

From the requirements. One `main.py` that holds the table, the checks, and every route is hard to change once login and a web page arrive. Split the jobs. The behaviour of the API stays the same.

1. One job per file

```
acmehr-api/
├── main.py                ← creates the app and starts the seed
├── database.py            ← URL, engine, session, get_db
├── models.py              ← Employee table
├── schemas.py             ← EmployeeCreate, EmployeeUpdate, EmployeeOut
├── routers/
│   └── employees.py       ← the routes
├── .env                   ← your machine. Not committed
├── .env.example           ← the names of the settings, committed
├── .gitignore             ← keeps .env, venv/, and test databases out of Git
├── requirements.txt
└── test_employees.py
```

The same files are in `examples/lesson07/`.



`database.py` is where the URL lives. Copy `.env.example` to `.env` and put your MySQL password in `.env` if it is not `password123`.

```
import os
from dotenv import load_dotenv
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

load_dotenv()

DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "mysql+mysqlconnector://trainee:password123@localhost:3306/training_db",
)
connect_args = {"check_same_thread": False} if DATABASE_URL.startswith("sqlite") else {}
engine = create_engine(DATABASE_URL, connect_args=connect_args)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

`yield` hands the session to the route and continues after the route finishes, which is where `close()` runs. You do not call `SessionLocal()` inside the route anymore.

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from database import get_db

router = APIRouter(prefix="/employees", tags=["employees"])

@router.get("", response_model=list[EmployeeOut])
def list_employees(department: str | None = None, db: Session = Depends(get_db)):
    query = db.query(Employee)
    if department is not None:
        query = query.filter(Employee.department == department)
    return query.all()
```

Piece	Job
`APIRouter`	A group of routes. `prefix` adds `/employees` so the path here is `""`
`tags`	The heading on `/docs`
`Depends(get_db)`	FastAPI calls `get_db`, passes you `db`, and closes it afterwards
`include_router`	In `main.py`: `app.include_router(employees_router)`

`pip install python-dotenv` so `load_dotenv()` reads `.env`. `.env` stays off GitHub. `.env.example` is the file you commit.

From this folder:

```
fastapi dev main.py
```

`/docs` still shows the employee routes. GET Ananya still returns **200**.

---

2. One test

`test_employees.py` points `DATABASE_URL` at a SQLite file before it imports the app, then calls the same routes. You do not need MySQL to see the test pass. The class app still uses MySQL when you run `fastapi dev`.

```
pip install pytest
pytest test_employees.py
```

Output:

```
3 passed
```

The tests check `/health`, Ananya's name, and a **404** for `E-9999`.

Common error: `ModuleNotFoundError: database`. The terminal is not in the folder that contains `main.py`. `cd` there, then `fastapi dev main.py`.

Common error: You committed `.env`. It holds the database password. The example folder already has a `.gitignore` that lists `.env`. Copy it into your project before your first commit.

Try these

1. **(Easy)** Open `/docs`. The employee routes sit under the tag **employees**.
2. **(Easy)** Run `pytest`. Three tests pass.
3. **(Medium)** Move `DATABASE_URL` only in `.env`, restart, and GET Ananya. The route file did not change.
4. **(Stretch)** Add `GET /employees/count` on the router that returns `{"count": n}`. It appears in `/docs` without a new `FastAPI()` line.

Interview: *What problem does Depends solve?* — The route asks for a session and does not open or close it by hand. Every route gets that the same way.

Key takeaways

- Table, checks, routes, and the URL live in different files.
- `APIRouter` groups the employee routes. `main.py` includes that router.
- `Depends(get_db)` opens the session and closes it after the response.
- `.env` is local. `.env.example` is shared. `pytest` checks the routes without MySQL.

Lab — Ticket ACME-207

Split `main.py` into this layout and add one test.

```
PYTHON/17_FastAPI/labs/L07_structure/
├── broken/      ← Part A. A split project with bugs.
├── starter/     ← Part B. The split files, with TODOs.
└── solution/    ← Open only after the checklist passes.
```

Each folder is a whole project. `cd` into it, copy `.env.example` to `.env`, then run `fastapi dev main.py` and `pytest test_employees.py` from there.

Part A — Fix the broken project (10 min)

`broken/` starts, and it has **3 bugs**:

1. `/docs` has no **employees** section, and `GET /employees/E-1042` is **404**.
2. After you fix 1, `GET /employees/E-1042` is still **404**. Read the path `/docs` shows for that route.
3. `pytest test_employees.py` uses MySQL, not a fresh SQLite file. No `test_acme.db` appears. Depending on your MySQL, the tests fail with `Can't connect` or `Access denied`, or they quietly run against your class database.

Part B — Build the split (15 min)

`starter/` has every file from section 1. Write `get_db`, move your four write routes from Lesson 6 into `routers/employees.py` with `Depends(get_db)`, include the router in `main.py`, and finish the two TODO tests.

Part C — Prove it works (5 min)

- [] `fastapi dev main.py` serves `GET /employees/E-1042`.
- [] `/docs` groups those routes under **employees**.
- [] `.env` is not committed. `.env.example` and `.gitignore` are.
- [] `pytest test_employees.py` prints `3 passed`.

Then do the same split in your own `acmehr-api` folder.

Homework

1. Push the split project to your own GitHub repo `acmehr-api`.
2. **Transfer drill (15 minutes).** Create `routers/health.py` with its own `APIRouter` and move `GET /health` into it. Remove the route from `main.py` and include the new router instead. The test for `/health` still passes.

Next: Authentication. A login route returns a token, and delete refuses anyone who is not HR.